

Polymer AI 逆向拆解与多 Agent 复现方案

— 现有 AI 产品的逆向拆解（功能·交互逻辑·可能实现方式） —

产品名称 Polymer AI（v3）

数据集 运营商电信营销数据集（72 条×28 字段）

岗位 产品经理

第一章 产品概述与选择理由	3
1.1 Polymer AI 是什么	3
产品定位	3
目标用户	3
核心能力	3
1.2 选择理由：为什么是 Polymer AI	3
1.3 核心功能地图（用户视角）	4
第二章 实际体验复盘（基于电信营销数据集）	5
2.1 数据集说明	5
2.2 上传 → 自动建表全过程（Polymer AI 操作复盘）	5
2.3 体验核心洞察	6
第三章 功能拆解：子任务识别与设计意图分析	8
3.1 拆解方法	8
3.2 完整用户旅程与子任务映射	8
3.3 八个子任务详解	8
T1 数据接入与文件解析	8
T2 字段识别与数据语义建模	9
T3 自动 Dashboard 生成	9
T4 自然语言意图理解（NLQ 核心）	10
T5 确定性查询与指标计算	10
T6 多轮上下文继承	10
T7 主动洞察与异常检测	11
T8 报告组织与结果交付	11
第四章 交互逻辑拆解：用户旅程与系统响应	13
4.1 完整用户旅程映射	13
4.2 交互设计亮点	13
4.3 多轮追问的交互逻辑详解	14

- 第五章 Agent / Skill 角色设计16
 - 5.1 Agent 与 Skill 的区分原则16
 - 5.2 完整 Agent / Skill 职责总表16
 - 5.3 Intent Agent 深度设计（最核心 Agent）17
 - 5.3.1 意图分类体系（5 类）17
 - 5.3.2 置信度机制与降级策略18
- 第六章 多 Agent 协作架构19
 - 6.1 协作模式选择：编排器模式19
 - 6.2 主流程：数据上传 → Dashboard 生成19
 - 6.3 生成 NLQ 流程：用户提问 → 图表返回20
 - 6.4 端到端场景（基于电信营销数据集）20
- 第七章 多 Agent 复现方案与与真实运行结果22
 - 7.1 复现目标与技术选型22
 - 7.2 核心伪代码22
 - 7.3 代码演示结果23
 - 7.4 关键设计原则（5 条）25
 - 7.5 本章小结25
- 第七章对思特奇营销评估助手的启示与优化建议26
 - 启示与优化建议26
- 附录26

第一章 产品概述与选择理由

1.1 Polymer AI 是什么

Polymer AI（polymersearch.com）是一款面向非技术用户的 AI 驱动数据可视化与智能分析平台，它的核心价值是帮助用户通过简单的对话和点击，快速连接、整合并分析来自不同来源的数据，从而生成洞察，辅助商业决策。其核心价值主张用一句话概括为：

把“上传数据 → 等 BI 团队 → 三天后拿到一张 PDF”这条链路，压缩为“上传 → 几秒出 Dashboard”。

产品定位

Polymer AI 定位于“轻量自助 BI”赛道，介于 Google Sheets（过于简单、无分析能力）和 Tableau/Power BI（专业门槛高、学习曲线陡）之间，特点是轻量、快速、易于分享，更侧重于快速探索与洞察，而非深度、精细的数据建模。

目标用户

没有 SQL 背景的营销人员、运营经理、数据分析师、销售团队、电商从业者、教育工作者以及需要快速制作报表的客户服务团队等，能够独立完成数据探索与洞察生成。

核心能力

能力维度	具体表现	传统 BI 工具对比
零代码接入	无需代码即可快速创建动态、可视化的仪表板	通常需要 IT 配置数据库连接
AI 自动理解	自动识别字段语义、数据类型、字段关系	需要人工定义维度和度量
自然语言查询	用口语提问即可生成图表，无需 SQL	需要 SQL 或拖拽配置

1.2 选择理由：为什么是 Polymer AI

本方案选 Polymer AI 主要因其“非技术用户可独立完成分析”的闭环特性与本次拆解目标（产品逻辑可复现）高度契合；若目标是企业级深度建模，Tableau / Power BI 仍更合适。

① 端到端的完整闭环

包含数据接入、字段识别、图表生成、自然语言分析、分享看板等闭环,平台提供了覆盖数据分析全链路的完整解决方案。

② 拆解难度适中，子任务边界清晰

Polymer AI 的功能模块划分自然、子任务独立性强，Polymer AI 能够让拆解结果直接落地为可运行代码。

③ 模拟数据集可验证

本方案使用模拟电信营销数据集（72 条，28 字段），覆盖 15 种策略类型、4 种渠道、18 个客群，可以在 Polymer

AI 上完整走一遍产品体验流程，确保拆解结论有数据支撑。

④ 深度贴合业务场景

针对营销活动评估这类典型场景，天然支持渠道归因、策略效果、客群画像、成本核算、转化漏斗、投诉分析等多维度数据交叉探究，让分析真正服务于决策。

1.3 核心功能地图（用户视角）

以下从用户能做什么（而非系统有什么）的视角，梳理 Polymer AI 的五大核心功能：

用户想完成的事	Polymer AI 对应能力	背后的 AI/系统能力
快速看懂一份表	上传后自动生成数据模型和推荐视图	字段类型识别、维度/指标判断、图表推荐
不用写 SQL 也能问数据	自然语言查询与图表返回	意图识别、实体抽取、查询生成
发现异常和机会点	自动洞察、排序、过滤、对比	统计分析、异常检测、业务规则解释
把结果发给别人	Dashboard 分享与嵌入	权限控制、报告结构化输出

Polymer AI 的本质是“把数据团队的工作封装成产品”：数据清洗、字段理解、图表选型、报告生成，每一步都原本需要专业人员完成，Polymer 用 AI 替代了这些中间环节。对于数据在系统里，但读不出洞察这一核心痛点，Polyme 的解决路径提供了一个可参考的产品范本。

第二章 实际体验复盘（基于电信营销数据集）

2.1 数据集说明

本次体验使用的数据集为运营商电信营销场景数据，字段覆盖完整的营销活动生命周期。数据结构适合验证 Polymer AI 是否真正能帮助业务人员完成“从数据到判断”的过程。

维度	详情
数据规模	72 条记录×28 个字段
时间范围	2026 年 5 月，按周划分（W1-W3）
策略类型	15 种（拉新、续约、加购、维系、融合营销、召回等）
渠道	4 种（SMS、App Push、Outbound Call、WeCom）
客群	18 个细分客群（大学生、高价值用户、银发用户等）
生命周期阶段	7 个阶段（新客、成长期、成熟期、活跃期、沉默期、流失风险、风险期）
地区	6 个区域（华北、华东、华南、华中、西南、西北）
关键指标字段	触达数、转化数、收入、成本、补贴成本、投诉数、满意度评分、风险等级

2.2 上传 → 自动建表全过程（Polymer AI 操作复盘）

本次体验使用一份电信营销模拟数据集，共 72 条营销活动记录、28 个字段，字段覆盖活动名称、策略类型、渠道、客群、触达量、转化量、收入、成本、投诉量、满意度等信息。该数据集能够模拟运营商营销评估场景，因此适合用于测试 Polymer AI 是否能完成“上传数据 → 自动分析 → 自然语言追问 → 输出洞察”的完整流程。上传 CSV 后，Polymer AI 能够自动读取表格结构，并进入数据分析界面。整个过程不需要手动建模，也不需要提前配置 SQL 或字段关系。

第一步，上传数据。用户只需将本地的 CSV 文件直接拖入网页界面，无需任何预处理。系统会立刻自动解析该文件，识别出其中包含的记录条数与字段数量（例如 72 条记录、28 个字段）。

我认为此步骤的设计目的是最大程度降低使用门槛，让用户无需先学习复杂的 BI 建模知识即可开始。

第二步，字段识别。在上传数据后，用户完全不需要手动配置字段类型。系统后端的 AI 引擎会自动分析并识别每个字段的语义，准确判断出哪些是日期、分类维度、数值指标或纯文本字段。

我认为这一步的核心目的在于，利用 AI 替用户完成对数据的初步理解与分类，省去了传统工具中繁琐的数据定义工作。

第三步，生成看板。当数据被理解后，系统会直接进入自动仪表板生成流程。用户会看到一个已经创建好的基础看板，上面直观展示了数据中的趋势、不同渠道表现、策略效果及排名等核心视图。

我认为这个设计的目的在于“先给用户一些可看的東西”，有效避免了从零开始创建时的“空白页焦虑”，让洞察立即可以看见。

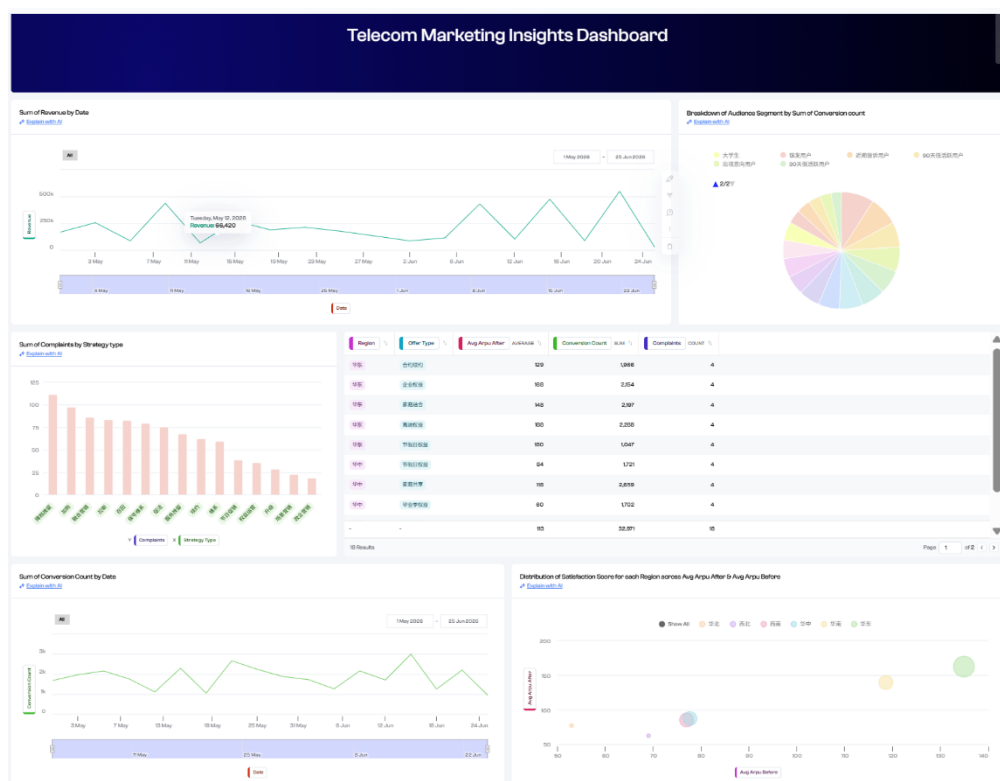
第四步，自然语言追问。基于已生成的看板，用户可以使用日常口语提出更具体的业务问题，例如“哪个渠道的ROI最高？”。系统会将这个自然语言问题实时翻译成聚合、排序等数据查询指令，并立即以图表形式返回答案。

我认为这一步把专业的分析动作翻译成了易理解的通用业务语言，真正实现了对话式分析。

第五步，洞察沉淀。在分析的最后，系统不会仅停留在展示图表，还会主动输出对数据的异常检测与智能建议，例如总结出表现最佳的渠道、风险较高的策略维度等结论。

我认为这使得工具的使用体验从单纯的“制作报表”迈向了“辅助决策”，帮助用户从数据中直接获得可采取行动的商业洞察。

系统的图表选型逻辑基本合理（时间序列→折线、分类汇总→柱状、占比→饼图），但对复合指标（如 $ROI = revenue / (cost + subsidy_{cost})$ ）无法自动生成，需要用户通过自然语言查询(Natural Language Query,NLQ)额外提问。



2.3 体验核心洞察

Polymer AI 的体验路径并不是“给用户一个万能聊天框”，而是先通过自动 Dashboard 建立整体认知，再通过自然语言问答支持局部探索，最后通过洞察解释帮助用户形成业务判断。这种路径符合业务人员的数据分析习惯：先看全局，再问重点，最后做决策。

洞察 1：Polymer 的竞争力不在于图表多，而在于让用户不需要思考下一步，每一步都有 AI 给出默认值。

洞察 2: NLQ 的本质是把 SQL 技能封装进产品, 但当前版本对复合指标仍需用户显式引导。

洞察 3: Polymer 最大的短板是缺乏持续监控式的主动洞察, 它不会在异常发生的第一时间主动推送预警, 仅在用
户发起分析时给出收尾总结。

第三章 功能拆解：子任务识别与设计意图分析

3.1 拆解方法

功能拆解的关键不是罗列页面按钮，而是回答：用户在什么场景下产生需求，系统替用户完成了哪些判断，这些判断如何组合成一个完整产品。

本报告采用“用户旅程逆向拆解法”：沿完整用户操作路径，在每一个交互节点提问，这里 AI 在做什么判断？输入是什么？输出是什么？从而将一个看似整体的产品逆向分解为边界清晰的子任务。

该方法的好处是：子任务来源于真实用户行为，而非技术架构，因此更容易映射到 Agent 设计，也更容易向非技术人员解释。

3.2 完整用户旅程与子任务映射

核心功能	用户价值	可拆子任务	可能实现方式
数据接入	无需建模即可开始分析	文件解析、字段类型识别、缺失值检查	pandas 读取文件 + schema profiler
语义建模	把字段变成业务可理解的维度/指标	指标识别、维度识别、时间字段识别	规则 + LLM 字段语义标注
自动 Dashboard	快速获得第一版分析入口	图表推荐、指标聚合、页面布局	图表模板库 + 推荐规则
自然语言提问	不用 SQL 也能完成查询	意图分类、实体抽取、查询生成	LLM 转结构化 JSON + DuckDB 执行
主动洞察	帮助用户发现没有主动提问的问题	异常检测、Top-N、趋势变化、风险提醒	统计规则 + 业务阈值 + LLM 解释
分享协作	把分析结果交付给团队	报告生成、权限控制、链接分享	报告模板 + 权限服务

3.3 八个子任务详解

T1 数据接入与文件解析

属性	内容
输入	原始 CSV / Excel / Google Sheets 文件
核心处理	文件格式识别、编码检测、表头识别、行列结构解析、空值/重复值初步检查

属性	内容
输出	可被系统读取的标准化数据表 + 基础数据质量报告
设计意图	让业务人员不需要先完成数据清洗、字段配置等专业步骤。用户只要上传文件，系统即可进入下一步。
技术实现	规则引擎识别表头、空值、重复行和异常格式；Python + pandas 读取
对应 Agent/Skill	Data Agent + File Parse Skill

T2 字段识别与数据语义建模

属性	内容
输入	已解析数据表的字段名、字段样本值、字段分布
核心处理	字段类型推断（数值/分类/日期/文本）；字段语义标注（维度/度量/时间轴/描述）；识别指标口径；建立可查询索引
输出	结构化数据模型 JSON：字段列表 + 类型标注 + 语义角色 + 可查询索引
设计意图	AI 读懂数据是整个系统的地基。只有字段语义正确，后续所有图表、NLQ 和洞察才能成立。传统 BI 工具把这一步交给用户手动配置，Polymer 把它自动化。
技术实现	LLM + few-shot 提示工程（输入字段名+样本值，输出结构化 JSON）；规则引擎兜底（日期正则/枚举值数量判断）
对应 Agent/Skill	Data Agent（LLM 标注部分）

T3 自动 Dashboard 生成

属性	内容
输入	结构化数据模型 JSON（T2 输出）
核心处理	根据字段特征选择图表类型（时间序列→折线、分类→柱状、占比→饼图）；选择高信息量字段组合；确定默认聚合方式；生成布局
输出	可交互的初始 Dashboard（含 3-5 个预建视图）
设计意图	"渐进式披露"：先给足够好的默认值，让用户从调整而非从零开始。将用户认知负担从"我要建什么图"转为"这张图够不够用"。
技术实现	规则引擎（数据类型→图表类型映射表）+ LLM 决策（多种图表都适合时选最优）

属性	内容
对应 Agent/Skill	AutoDash Agent

T4 自然语言意图理解（NLQ 核心）

属性	内容
输入	用户自然语言问题 + 数据模型上下文 + 上一轮意图（Memory Agent 提供）
核心处理	意图分类（聚合/筛选/排序/趋势/对比）；实体抽取（字段名/时间范围/筛选条件/聚合方式）；歧义消解；置信度评估
输出	结构化 intent JSON：{intent_type, group_by, metric, filters, time_range, sort, chart_type, confidence}
设计意图	NLQ 的本质是"把 SQL 技能封装进产品"。这一环节直接决定用户体验上限：理解准确则用户感到流畅；理解偏差则用户对 AI 失去信任。
置信度机制	≥0.85 直接执行；0.60-0.85 执行并追问确认；<0.60 给出 2-3 个可能解读让用户选择
技术实现	LLM + 系统提示中注入数据模型 Schema 的 system 提示；上下文管理
对应 Agent/Skill	Intent Agent（LLM 核心）+ Memory Agent（上下文）

T5 确定性查询与指标计算

属性	内容
输入	Intent JSON + 原始 DataFrame + 指标计算规则
核心处理	数据筛选、分组聚合、指标公式计算、排序、Top-N 返回、图表数据生成
输出	结果 DataFrame + 图表配置 + 可复核指标值
设计意图	计算部分必须稳定、可验证，不能交给 LLM 自由生成。LLM 负责理解和解释，确定性 Skill 负责计算，两者分工是多 Agent 设计的核心原则。
技术实现	pandas / DuckDB 执行聚合查询；指标公式规则库（ROI=revenue/(cost+subsidy)）
对应 Agent/Skill	Query Skill（确定性，无 LLM）

T6 多轮上下文继承

属性	内容
输入	当前用户问题 + 上一轮 intent JSON + 历史问答记录
核心处理	判断当前问题是否为追问；继承上一轮分析维度；修改局部指标或筛选条件；更新对话状态
输出	新一轮 intent JSON + 更新后的 conversation_state
设计意图	多轮交互的核心不是展示多条聊天气泡，而是系统能把上一轮结构化意图保存下来，在下一轮短句中复用。例如"那投诉风险呢？"本身没有说明分析对象，系统必须从上一轮读取 channel 维度。
技术实现	Memory Agent 保存 last_intent；Intent Agent 结合 previous_intent 解析当前问题
对应 Agent/Skill	Memory Agent + Intent Agent 协作

T7 主动洞察与异常检测

属性	内容
输入	全量数据 + 指标结果 + 业务规则/阈值
核心处理	统计检测（Z-score/同比变化率）；识别异常指标；生成自然语言洞察摘要；推送给用户
输出	洞察卡片："Outbound Call 投诉量是均值的 2.7 倍，建议复盘话术策略"
设计意图	从"被动响应"到"主动推送"的关键跃迁。这是 Copilot 与 Chatbot 的本质区别。Polymer 当前版本此能力较弱。
技术实现	统计规则引擎（阈值检测）+ LLM（将异常数据转化为自然语言描述）；后台定时任务
对应 Agent/Skill	Insight Agent

T8 报告组织与结果交付

属性	内容
输入	图表结果 + 洞察结论 + 用户追问历史
核心处理	组织分析结论；生成摘要；整理图表和表格；保留证据链；生成可分享内容
输出	Dashboard 看板 + 分析报告摘要 + 导出文件 + 分享链接
设计意图	把一次探索过程沉淀为可复用的分析资产，让结果能被团队协作、复盘和继续迭代。结论必须关联指标、图表和数据口径，不能只是文字结论。

属性	内容
技术实现	Report Agent + 模板生成；HTML/PDF/Word 导出；权限与分享链接管理
对应 Agent/Skill	Report Agent

第四章 交互逻辑拆解：用户旅程与系统响应

Polymer AI 的交互逻辑可以概括为“系统先主动给结构，用户再追问细节”。这比传统 BI 的空白画布更适合非技术用户，因为用户不需要先知道自己该拖哪个字段、选哪个图表。

4.1 完整用户旅程映射

用户阶段	用户心理	关键交互	系统应完成的隐性判断
进入产品	我能不能直接用？	上传/拖拽数据入口	判断文件格式、字段数量、编码与数据质量。
初识数据	这份表讲的是什么？	自动字段列表和推荐视图	判断哪些字段是维度、指标、时间轴。
形成问题	哪里表现最好/最差？	搜索框或 AI 问答框	识别用户意图、指标口径、筛选条件。
获得答案	这个结论可信吗？	图表 + 排序 + 简短解释	生成可验证查询结果并展示计算口径。
继续追问	为什么会这样？	多轮追问、过滤、钻取	继承上下文，缩小分析范围。
交付结果	怎么发给别人？	保存看板、导出、分享	把探索过程沉淀为报告结构。

4.2 交互设计亮点



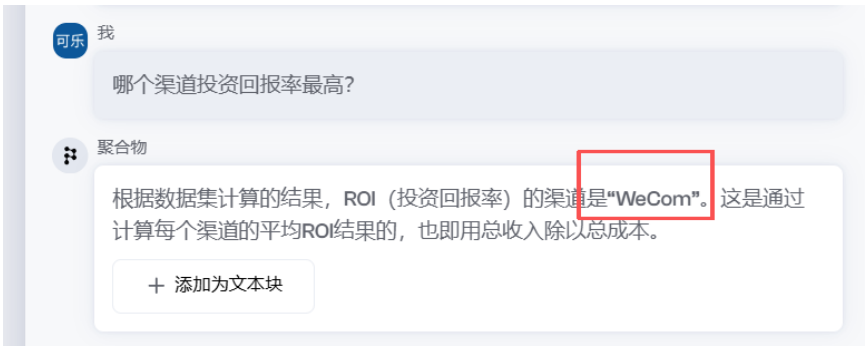
Polymer AI 没有把 AI 做成孤立聊天框，而是把 AI 嵌入数据分析流程：上传时 AI 识别字段，看板中 AI 推荐视图，追问时 AI 理解问题，输出时 AI 组织解释。产品价值来自“AI 被放在正确的业务节点上”。

4.3 多轮追问的交互逻辑详解

多轮追问是 Polymer AI 区别于普通图表工具的核心能力，其关键不是页面上出现多条聊天气泡，而是系统能把上一轮的结构化意图保存下来，并在下一轮的短句中复用：

轮次	用户输入	Memory Agent 读取	Intent Agent 解析	系统输出
Turn 1	哪个渠道 ROI 最高？	无历史（首轮）	group_by=channel; metric=roi; sort=desc	WeCom ROI 最高
Turn 2	那投诉风险呢？	继承 channel 维度	group_by=channel; metric=complaint_rate	外拨电话投诉率最高
Turn 3	换成策略维度看 ROI	用户显式切换维度	group_by=strategy_type; metric=roi	政企营销 ROI 最高
Turn 4	转化率最高的是谁？	继承 strategy_type	group_by=strategy_type; metric=conversion_rate	用户投诉安抚策略转化率最高

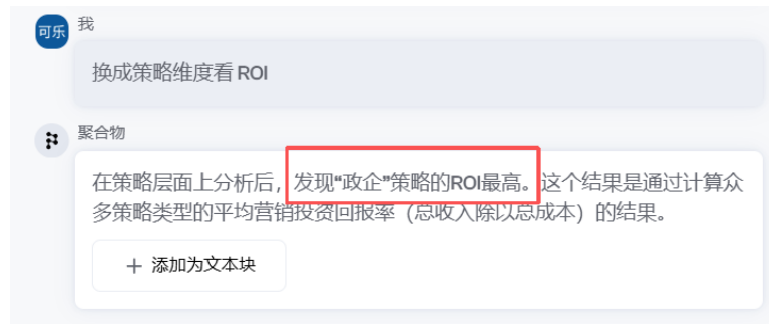
Turn 1



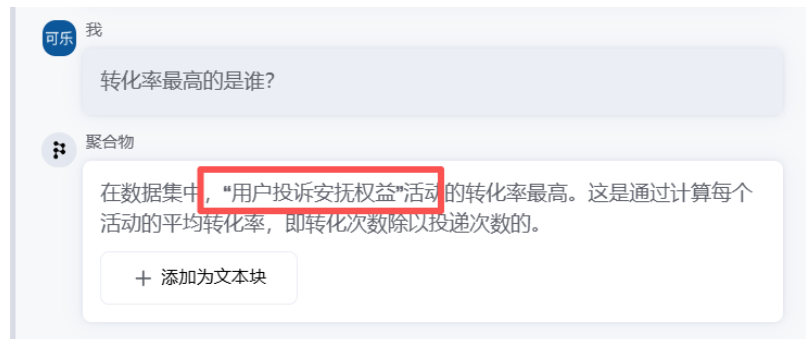
Turn 2



Turn 3



Turn 4



第五章 Agent / Skill 角色设计

5.1 Agent 与 Skill 的区分原则

采用“用户旅程逆向拆解法”：沿用户每一步操作追问三个问题：这里系统接收了什么输入？AI 做了什么判断？输出如何影响下一步交互？由此把一个完整产品拆成可复现的子任务。

Agent（自主决策）	Skill（确定性工具）
需要调用 LLM 进行推理判断	输入输出关系固定，不需要 LLM
输出结果不完全确定	规则/代码即可实现，结果稳定可复核
处理模糊输入、需要理解语义	处理计算、查询、格式化等任务
示例：Intent Agent、Insight Agent	示例：Query Skill、File Parse Skill

在本方案中，Agent 和 Skill 有明确区分：

- Agent：具备自主决策能力，需要调用 LLM 进行推理判断，输出结果不完全确定性
- Skill：确定性工具，输入输出关系固定，无需 LLM，可用规则/代码直接实现

5.2 完整 Agent / Skill 职责总表

本方案共设计 6 个业务 Agent + 1 个编排器 Orchestrator（合计 7 个 Agent 节点）+ 2 个 Skill（Query Skill、File Parse Skill），覆盖 Polymer AI 的 8 个核心子任务。

角色名称	类型	对应子任务	核心职责	输入	输出
Orchestrator	Agent	全局编排	识别当前流程阶段，路由到对应 Agent/Skill，整合结果	用户动作 + 数据模型 + 上下文	执行计划 + 最终响应
Data Agent	Agent	T1 + T2	解析文件，识别字段类型，用 LLM 完成字段语义标注	CSV 文件路径	schema JSON
AutoDash Agent	Agent	T3	根据 schema 推荐图表类型，生成首屏 Dashboard 布局	schema JSON	dashboard config

角色名称	类型	对应子任务	核心职责	输入	输出
Memory Agent	Agent	T6	保存上一轮意图，为短句追问提供上下文	历史 intent + 对话记录	previous_intent JSON
Intent Agent	Agent	T4	结合当前问题与 previous_intent，识别意图/实体/置信度	用户问题 + previous_intent	intent JSON
Query Skill	Skill	T5	执行聚合、过滤、排序、指标公式计算（纯代码，无 LLM）	intent JSON + DataFrame	结果表 + 图表数据
Insight Agent	Agent	T7	检测异常指标，用 LLM 生成自然语言洞察和行动建议	指标结果 + 业务规则	洞察卡片
Report Agent	Agent	T8	把图表、结论、建议组织成可分享的结构化报告	图表 + 洞察 + 对话上下文	报告摘要 + 导出内容
File Parse Skill	Skill	T1	文件格式识别、编码检测、表头与行列解析（确定性）	原始文件	标准化数据表

5.3 Intent Agent 深度设计（最核心 Agent）

5.3.1 意图分类体系（5类）

意图类型	典型问题	SQL 模式	电信营销示例
聚合查询	各渠道转化率分别是多少？	SELECT dim, AGG(metric) GROUP BY dim	SELECT channel, SUM(conversion)/SUM(delivered)
筛选查询	高风险的活动有哪些？	SELECT * WHERE condition	SELECT * WHERE risk_level="高"
排序排行	ROI 最高的前 5 个策略	SELECT dim ORDER BY metric LIMIT N	SELECT strategy_type ORDER BY roi DESC LIMIT 5
趋势分析	过去三周收入趋势	SELECT time, AGG(metric) GROUP BY time	SELECT week, SUM(revenue) GROUP BY week
对比分析	SMS 和 WeCom 的综合对比	SELECT dim, m1, m2 WHERE dim IN (...)	SELECT channel, roi, complaint_rate WHERE channel IN (...)

5.3.2 置信度机制与降级策略

置信度	处理策略	用户体验	示例场景
≥ 0.85	直接执行查询，返回结果	用户无感知，体验流畅	"各渠道转化率" → 直接执行
0.60–0.85	执行查询，同时显示"我理解您问的是...是否正确？"	用户一键确认或纠正	"划算的渠道" → 执行 ROI 排名并追问
< 0.60	触发澄清追问，给出 2-3 个可能解读	用户选择后重新执行	"最好的" → 问：是 ROI 最高/转化率最高/投诉最少？

第六章 多 Agent 协作架构

6.1 协作模式选择：编排器模式

本方案选用"编排器模式"（Orchestrator Pattern）：由一个中央 Orchestrator 负责任务路由、状态管理和结果整合，各 Agent/Skill 只处理自己的专属任务。

对比维度	编排器模式（本方案）	点对点通信模式
任务路由	集中在 Orchestrator，逻辑清晰	分散在各 Agent，难以维护
错误处理	Orchestrator 统一捕获，支持重试	错误传播路径复杂
状态管理	Memory Agent 和 Orchestrator 共同维护上下文，便于追踪	状态分散，难以追踪
扩展性	新增 Agent 只需注册到 Orchestrator	新增 Agent 需修改多处逻辑
可解释性	每一步都有执行轨迹，适合展示	执行过程不易展开说明

6.2 主流程：数据上传 → Dashboard 生成

主流程对应 Polymer AI 的第一段体验：用户上传数据后，系统自动理解数据结构，并生成初始分析看板。

- 1.用户上传 CSV，Orchestrator 接收文件路径
- 2.Orchestrator 调用 Data Agent
- 3.Data Agent 读取数据，识别字段类型与业务语义。
- 4.Data Agent 输出输出 schema JSON，包括字段名、字段类型、语义角色。
- 5..Orchestrator 调用 AutoDash Agent
- 6.AutoDash Agent 根据 schema 自动生成 Dashboard 配置。
- 7.前端渲染核心指标卡、渠道 ROI 图、策略 ROI 图。
- 8.Orchestrator 异步触发 Insight Agent
- 9. Insight Agent 扫描异常指标，如高投诉渠道、低满意度策略
- 10.将洞察卡片推送到 Dashboard

主流程输出

输出内容	示例（真实值）
核心指标卡	总触达 1,613,831；总转化 58,243；整体转化率 0.036；整体 ROI 2.56
推荐图表	渠道 ROI 排名、策略 ROI 排名、风险等级分布 渠道 ROI 排名：WeCom 3.10 / SMS 2.85 / App Push 2.79 / Outbound 0.61
初始洞察	WeCom ROI 较高；Outbound Call 投诉风险较高（0.0200）

数据模型	字段类型、指标字段、维度字段、时间字段
------	---------------------

6.3 生成 NLQ 流程：用户提问 → 图表返回

NLQ，即 Natural Language Query，自然语言问数。这一流程对应用户在工作台中输入问题。例如：哪个渠道 ROI 最高？

- 1.用户输入自然语言问题
- 2.Orchestrator 接收问题文本 + 当前 schema
- 3.Orchestrator 调用 Memory Agent → 获取 previous_intent
- 4.Orchestrator 调用 Intent Agent（输入：问题 + previous_intent + schema）→ 输出 intent JSON + 置信度
- 5.若置信度 < 0.60，Orchestrator 返回澄清追问，等待用户补充后重新执行
- 6.置信度达标后，Orchestrator 调用 Query Skill → 执行查询，返回 DataFrame + 图表数据
- 7.Orchestrator 调用 Insight Agent，对结果生成业务解读
- 8.Orchestrator 调用 Report Agent，组织最终回答（图表 + 结论 + 建议）
- 9.Memory Agent 记录本轮 intent，供下一轮追问使用

6.4 端到端场景（基于电信营销数据集）

场景：营销经理问"哪个渠道在华东最划算？"

步骤	执行主体	输入	处理内容	输出
1	Orchestrator	用户问题	路由到 Memory Agent + Intent Agent	—
2	Memory Agent	历史对话	首轮无历史，返回空 previous_intent	previous_intent=null
3	Intent Agent	问题+"华东","划算"	意图=排序；维度=channel；指标=roi；筛选=region="华东"；置信度=0.98（"划算"→roi，直接执行）	intent JSON + confidence=0.98
4	Orchestrator	confidence=0.98	≥0.85，直接执行（未触发澄清）	执行查询
5	Query Skill	intent JSON	SELECT channel, (SUM(revenue))/(SUM(cost)+SUM(subsidy_cost)) AS roi FROM df WHERE region='华东' GROUP BY channel ORDER BY roi DESC	4 行×2 列 DataFrame
6	Insight Agent	DataFrame	检测：WeCom ROI 最高	洞察卡片
7	Report Agent	图表+洞察	组织输出：图表 + "华东 WeCom 综合效能最	最终回答

步骤	执行主体	输入	处理内容	输出
			优，建议优先配置"+ 数据证据	
8	Memory Agent	本轮 intent	保存本轮分析维度和指标	供下一轮追问使用

真实计算结果（华东，按 ROI 降序）：

channel	roi
WeCom	5.00
SMS	4.50
App Push	4.05
Outbound Call	0.77

建议：华东地区 WeCom 综合效能最优（ROI=5.00），建议优先配置资源；Outbound Call 投诉率最高（0.0176），建议复盘话术。

第七章 多 Agent 复现方案与与真实运行结果

7.1 复现目标与技术选型

复现目标：实现上传营销数据 → 自动 Dashboard 生成 → 自然语言追问 → 多轮上下文继承 → 图表结果 + 营销优化建议的完整体验闭环（原型演示级别）。复现重点不是完整复制 Polymer AI 的商业化功能，而是证明其核心产品逻辑可以被拆成多个 Agent / Skill，并通过协作方式实现。

技术选型：为保证演示真实有效，Intent Agent 真实调用 **gpt-4o** 端点（非规则模拟），Query Skill 用 DuckDB 做确定性计算，结果可复核。

组件	选型	选型理由
原型语言	Python	便于快速处理数据、实现 Agent 类和命令行交互
数据处理	Python + pandas	覆盖数据清洗、聚合、指标计算
查询计算	Query Skill	负责确定性聚合、排序、指标计算
查询引擎	DuckDB（内存数据库）	直接对 DataFrame 执行标准 SQL，无需配置数据库服务器
可视化	matplotlib / plotly	matplotlib 静态图表；plotly 交互图表
前端演示	HTML	用于展示工作台、多轮对话、上下文状态和执行轨迹
Agent 编排	纯 Python 类调用	原型阶段无需引入复杂框架，逻辑清晰易读易验证
大模型能力	gpt-4o（真实调用）	Intent Agent 真实解析自然语言；端点 https://xiaoai.plus/v1/chat/completions

7.2 核心伪代码

```
# 多 Agent 协作主流程伪代码
# 调度者 Orchestrator 维护共享黑板 Blackboard，在 Agent 之间传递任务与状态

# 阶段一：数据上传与建模（DataAgent 与 MemoryAgent 并行）
dataset = DataAgent.profile(csv_file)    # 并行
memory = MemoryAgent.init()             # 并行
dashboard = AutoDashAgent.build(dataset.schema)    # 串行：生成推荐看板

# 阶段二：多轮自然语言问数（支持上下文继承）
while user_question:
    previous_intent = memory.recall()      # 并行：预取上下文
    intent = IntentAgent.parse(user_question, dataset.schema, previous_intent) # 串行：解析意图

    if intent.confidence < 0.60:
        return ClarificationQuestion(intent.candidates)    # 异常：低置信 -> 澄清追问

    result = QuerySkill.run(intent, dataset.dataframe)    # 并行
    AutoDashAgent.refresh(intent)                        # 并行
    if result is EMPTY:
```

```

    result = QuerySkill.retry(intent, relax=True)      # 异常：无命中 -> 放宽过滤重试

    insight = InsightAgent.explain(result, business_rules)  # 串行：AI 解读
    if Reviewer.reject(insight):
        return RetryWithHint(insight.note)            # 评审者把关：打回重算

    response = ReportAgent.compose(result.chart, insight, action_suggestion)
    memory.remember(user_question, intent, response)    # 状态同步：保存供下轮继承
    return response

```

7.3 代码演示结果

...

阶段一：数据上传与自动建模

数据集: 72 行 × 28 字段

Schema(前 6 字段):

```

- activity_id    type=text    role=description sample=['A001', 'A002', 'A003']
- week          type=category role=dimension sample=['W1', 'W3', 'W1']
- region        type=category role=dimension sample=['西北', '华中', '华中']
- channel       type=category role=dimension sample=['App Push', 'App Push', 'Outbound Call']
- strategy_type type=category role=dimension sample=['权益升级', '用户投诉安抚策略', '维系']

```

AutoDash 推荐视图:

```

[line] 收入趋势（按周）
[bar] 各渠道 ROI 排名
[bar] 各策略 ROI 排名
[pie] 风险等级分布

```

阶段二：多轮自然语言问数（Memory Agent 上下文继承）

--- Turn 1 | 用户: 哪个渠道 ROI 最高? ---

Intent JSON: {"intent_type": "sort", "group_by": "channel", "metric": "roi", "filters": [], "time_range": null, "sort": "desc", "chart_type": "bar", "confidence": 0.98}

SQL: SELECT channel, (SUM(revenue)) / (SUM(cost) + SUM(subsidy_cost)) AS roi FROM df GROUP BY channel ORDER BY roi DESC

```

channel    roi
WeCom 3.097293
SMS 2.849434
App Push 2.789507

```

Outbound Call 0.610165

洞察: [channel] WeCom 的 roi 最高(3.097); Outbound Call 最低(0.610)。

--- Turn 2 | 用户: 那投诉风险呢? ---

Intent JSON: {"intent_type": "sort", "group_by": "channel", "metric": "complaint_rate", "filters": [], "time_range": null, "sort": "desc", "chart_type": "bar", "confidence": 0.98}

SQL: SELECT channel, (SUM(complaint_count)) / (SUM(reach)) AS complaint_rate FROM df GROUP BY channel ORDER BY complaint_rate DESC

```
channel complaint_rate
Outbound Call    0.020037
App Push         0.003692
SMS              0.003071
WeCom            0.002935
```

洞察: [channel] Outbound Call 的 complaint_rate 最高(0.020); WeCom 最低(0.003)。

--- Turn 3 | 用户: 换成策略维度看 ROI ---

Intent JSON: {"intent_type": "sort", "group_by": "strategy_type", "metric": "roi", "filters": [], "time_range": null, "sort": "desc", "chart_type": "bar", "confidence": 0.98}

SQL: SELECT strategy_type, (SUM(revenue)) / (SUM(cost) + SUM(subsidy_cost)) AS roi FROM df GROUP BY strategy_type ORDER BY roi DESC

```
strategy_type    roi
政企营销 3.694320
老带新 2.716348
用户投诉安抚策略 2.369349
... (其余 12 类略, 融合营销 0.243 最低)
```

洞察: [strategy_type] 政企营销 的 roi 最高(3.694); 融合营销 最低(0.243)。

--- Turn 4 | 用户: 转化率最高的是谁? ---

Intent JSON: {"intent_type": "sort", "group_by": "strategy_type", "metric": "conversion_rate", "filters": [], "time_range": null, "sort": "desc", "chart_type": "bar", "confidence": 0.98}

SQL: SELECT strategy_type, (SUM(conversion)) / (SUM(reach)) AS conversion_rate FROM df GROUP BY strategy_type ORDER BY conversion_rate DESC

```
strategy_type    conversion_rate
用户投诉安抚策略 0.124263
老带新          0.064500
政企营销        0.052049
... (其余略)
```

洞察: [strategy_type] 用户投诉安抚策略 的 conversion_rate 最高(0.124); 续约 最低(0.032)。

=====
阶段三: 端到端场景 (华东最划算? 真实 LLM 解析 + 澄清 + 计算 + 解释)
=====

用户: 哪个渠道在华东最划算?

置信度=0.98 -> 直接执行

Intent JSON: {"intent_type": "sort", "group_by": "channel", "metric": "roi", "filters": [{"field": "region", "op": "=", "value": "华东"}], "time_range": null, "sort": "desc", "chart_type": "bar", "confidence": 0.98}

SQL: SELECT channel, (SUM(revenue)) / (SUM(cost) + SUM(subsidy_cost)) AS roi FROM df WHERE region = '华东' GROUP BY channel ORDER BY roi DESC

```
channel    roi
WeCom 5.001497
SMS 4.502510
App Push 4.049059
Outbound Call 0.770658
```

洞察: [channel] WeCom 的 roi 最高(5.001); Outbound Call 最低(0.771)。

建议: 华东地区 WeCom 综合效能最优 (ROI=5.001), 建议优先配置资源; 投诉最高渠道为 Outbound Call (投诉率=0.0176), 建议复盘话术。

DONE.

...

7.4 关键设计原则（5条）

- 1.LLM 负责"理解与解释", 确定性 Skill 负责"计算与查询", 两者分工严格, 指标计算不可控风险为零
- 2.多轮交互必须有 Memory Agent 保存结构化 intent, 而非只保留聊天气泡文本
- 3.所有 Agent 输出结构化 JSON（schema/intent/dashboard config），便于调试与复用
- 4.置信度机制：低置信不强行回答，返回澄清问题，保护用户信任
- 5.报告生成保留证据链：结论必须关联指标、图表和数据口径，不能只输出文字结论

7.5 本章小结

通过以上复现方案，可以证明 Polymer AI 的核心能力能够被拆解为一组可协作的 Agent / Skill：

Data Agent 负责读懂数据

AutoDash Agent 负责生成首屏看板

Memory Agent 负责多轮上下文继承

Intent Agent 负责理解自然语言问题

Query Skill 负责稳定计算

Insight Agent 负责解释业务含义

Report Agent 负责组织最终输出

Orchestrator 负责统一编排

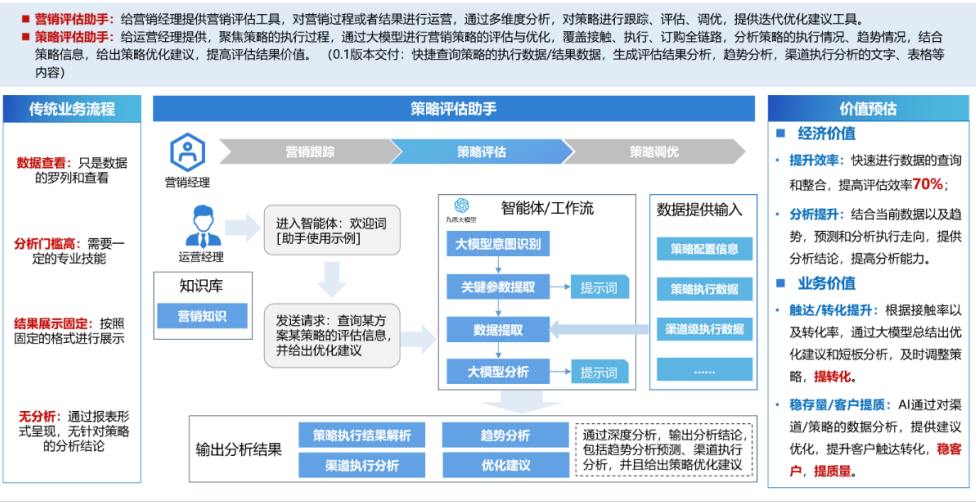
这套设计不仅复现了 Polymer AI 的核心体验，也能迁移到思特奇营销评估助手场景中，用于支持“察策打评”中的评估、诊断和策略优化环节。

第七章对思特奇营销评估助手的启示与优化建议

启示与优化建议

产品方案

营销评估助手@策略评估助手V0.1



从图中可以看到，思特奇当前的营销评估助手已经明确识别了传统业务流程的痛点：数据查看只是数据罗列和查看；分析门槛高，需要专业技能；结果展示固定，按照报表形式呈现；缺少分析，只给结果，不给策略解释。

当前方案已经通过“大模型意图识别 → 关键参数提取 → 数据提取 → 大模型分析 → 策略执行结果解析 / 趋势分析 / 渠道执行分析 / 优化建议”形成了完整链路。这条链路适合处理一次性查询，但真实运营场景往往不是一次问完，而是连续追问：因此，建议在营销评估助手中增加 **Memory Agent/上下文记忆能力**。

当前状态系统可以回答：这个策略执行得怎么样？哪个渠道效果好？趋势如何？有什么优化建议？下一步可以把这个闭环做得更产品化：不仅评估当前策略，还要把评估结果反哺下一轮策略生成。下一轮应该怎么投？哪些客群应该继续触达？哪些渠道应该降频？哪些权益应该替换？预算应该如何重新分配？

输入	处理	输出
当前策略执行数据	计算 ROI、转化率、投诉率、满意度	当前策略评分
渠道级执行数据	比较各渠道表现	渠道调整建议
客群级响应数据	判断高响应客群和低响应客群	客群筛选建议
历史策略数据	查找相似成功策略	复用策略模板
业务约束	预算、频次、投诉限制	下一轮策略方案

附录

- Polymer AI 官网：<https://www.polymersearch.com>

- Polymer AI 体验页面: <https://v3.polymersearch.com>
- 本报告数据: 运营商电信营销模拟数据集 polymer_ai_rich_telecom_marketing_dataset.csv
- 思特奇 AI 产品页面: <https://ai.si-tech.com.cn/operator/aiHome>

— 文档结束 —